

帮我重构...

修个 bug...

帮我重构...

修个 bug...

修个 bug...

修个 bug...

修个 bug...

# 从魔法到工程：Coding Agent 落地实践指南

揭秘 OpenCode 的系统架构、工作流与可复用资产沉淀

帮我重构...

帮我重构...

修个 bug...

修个 bug...

修个 bug...

帮我重构...





# Coding Agent 不是万能聊天框，而是本地运行的工程系统

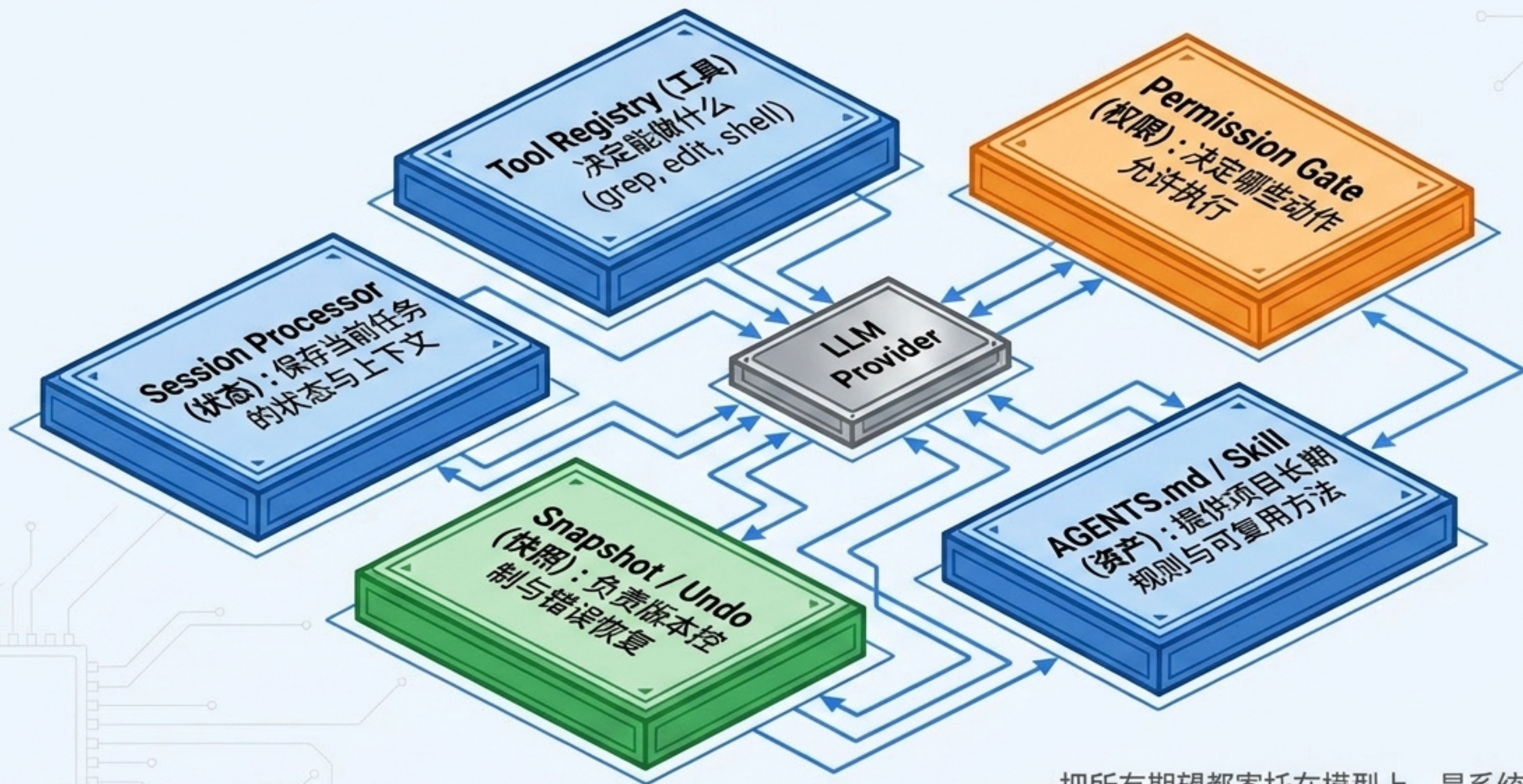
|      | 🗨️ Web Chatbot | ⚙️ Local Agent Runtime |
|------|----------------|------------------------|
| 核心定位 | 一次性问答工具        | 持续的决策与执行闭环（ReAct）      |
| 交互载体 | 孤立的网页对话框       | 驻留在终端、本地项目与 Git 仓库     |
| 动作空间 | 仅生成文本代码        | 阅读、搜索、编辑、执行命令、运行测试     |
| 成功标准 | 回答看起来合理        | 动作未越权、代码通过验证、失败可恢复     |

核心洞察：Agent 的核心价值在于**执行闭环**，而非单纯的思考过程。必须将其置于**受控、可观察、可验证**的工作流中。



# 稳定性来自模块的有机组合，而非万能提示词

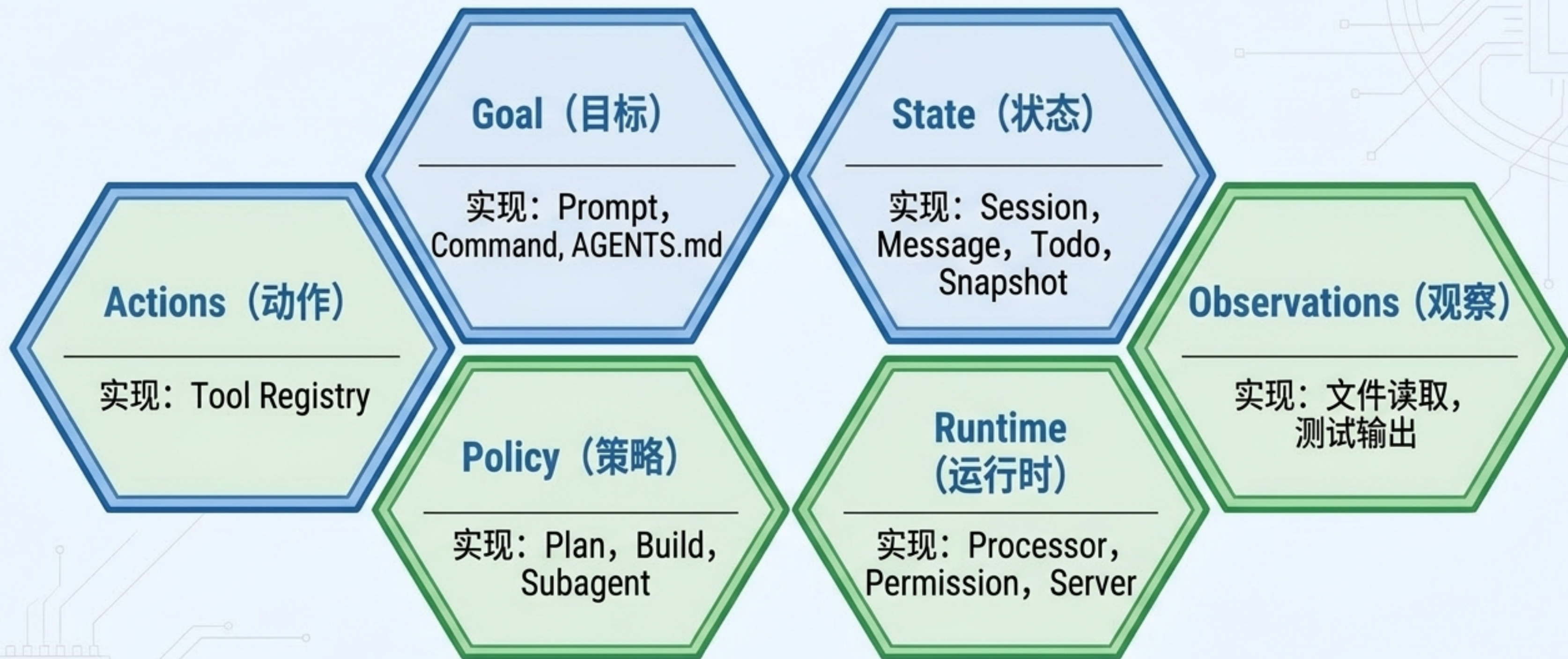
OpenCode = TUI / CLI / Server + 核心组件群



把所有期望都寄托在模型上，是系统设计失败的开始。



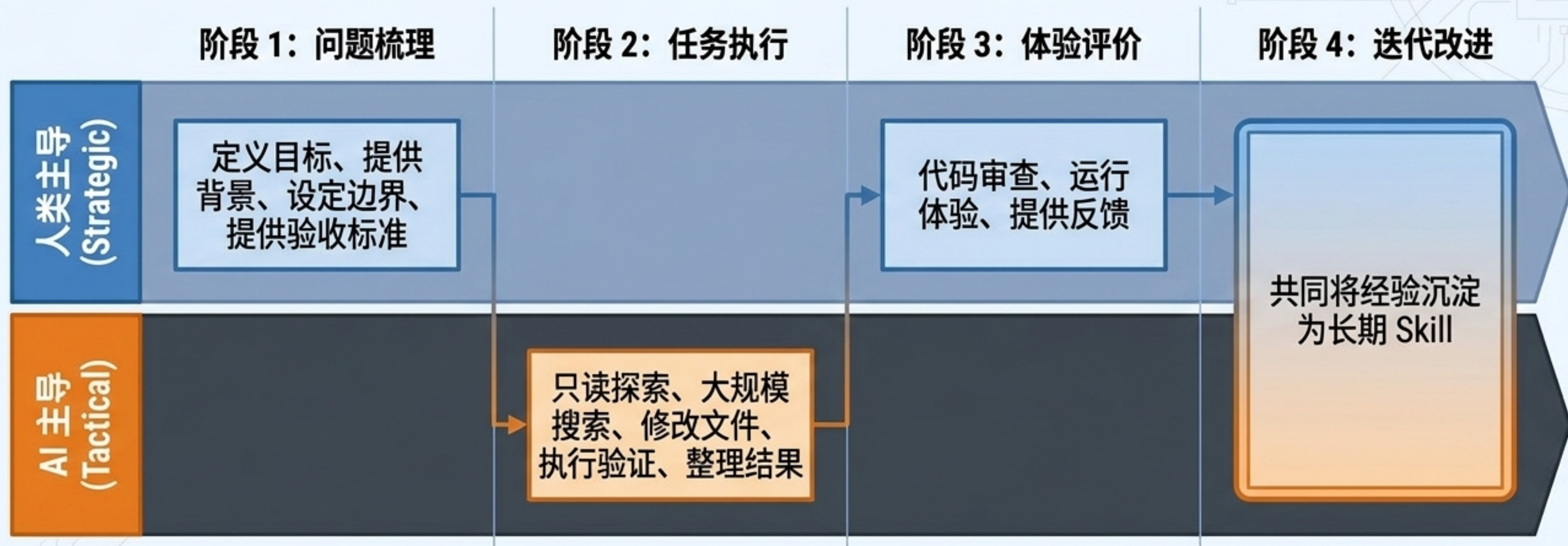
# 系统化控制 Agent 的 6 个工程抽象槽位



**排错指南:** 模型不是 Agent 的全部。修复 Bug 时, 先检查是哪个槽位输入了劣质信息。



# 人机协作的新分工：人类布置环境，AI 负责闭环



## 隐喻：导演与执行团队

导演不需要亲自写每一行代码，但必须明确剧本背景，并判断最终成果是否合格。



# 拒绝模糊指令：将一句话需求转化为结构化任务卡

帮我重构这个模块。

**错误：AI 在猜你的意图（缺失上下文）**



## Task Card

**动机 & 目标：**为什么做？当前问题是什么？最终交付什么？

**背景 & 输入：**业务规则是什么？涉及哪些文件和数据口径？

**工具：**可以调用哪些脚本、API 和数据服务？

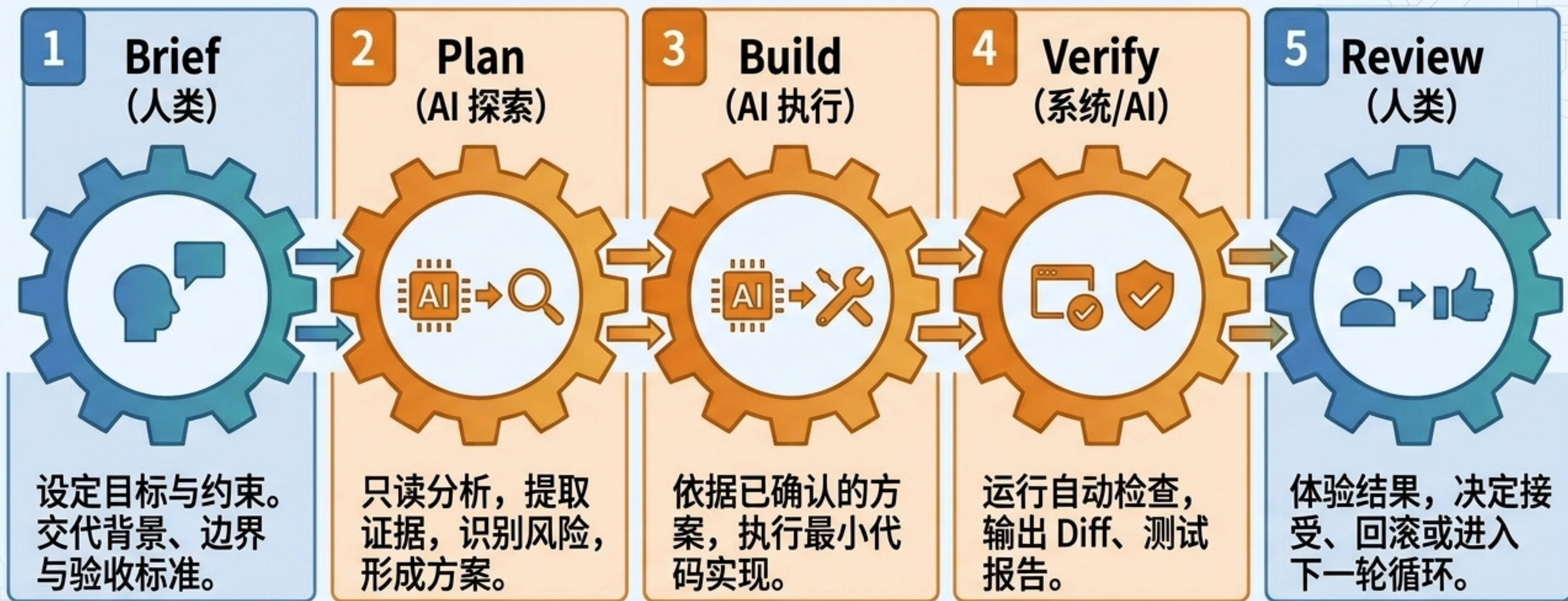
**边界：**哪些文件绝对不能改？（明确禁区）

**流程 & 验收：**先分析还是直接改？什么条件算真正完成？

**好 Prompt 的本质不是更长或更花哨，而是为 AI 搭建一个无歧义的可执行环境。**



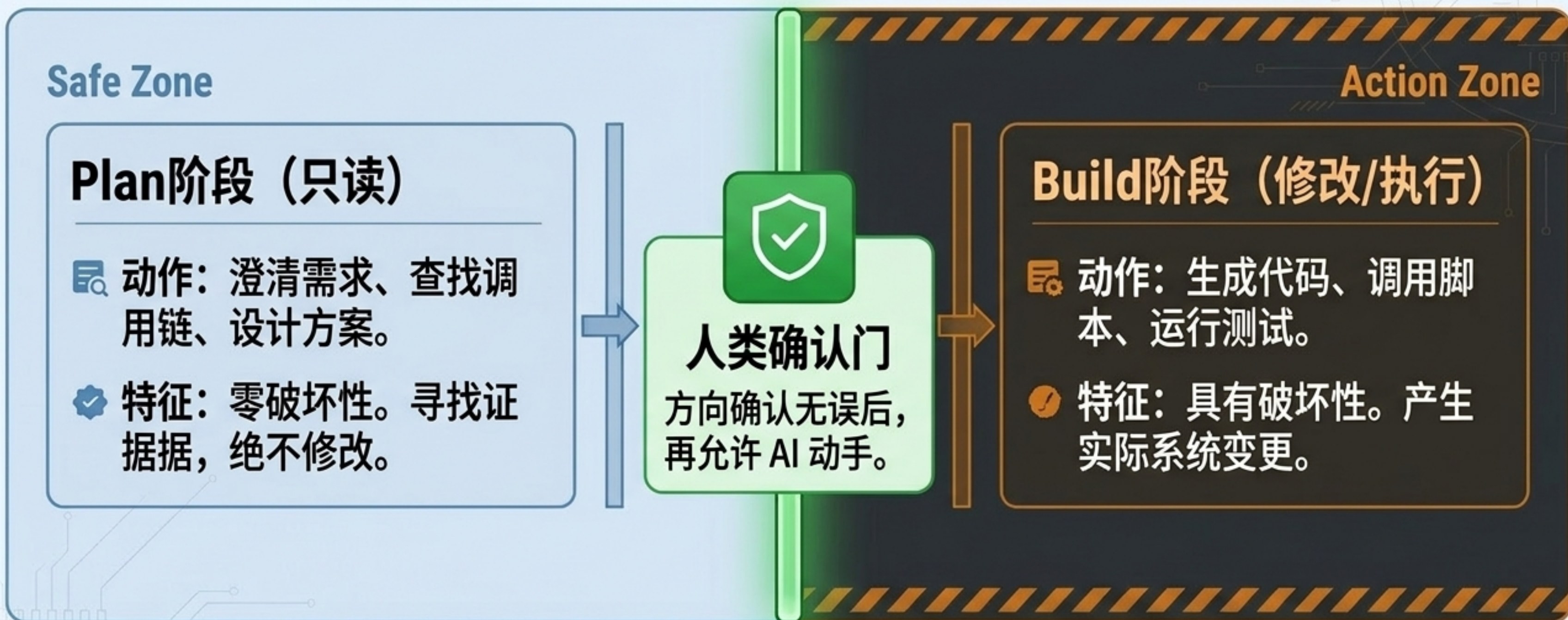
# 黄金五步工作流：先说清楚再执行，执行必取证



注：此工作流不仅适用于写代码，同样适用于数据分析、系统迁移与故障排查。



# Plan 与 Build 分离：降低错误成本的最重要拦截点



**核心洞察：相信 AI 的修改能力，但警惕其盲目的方向感。先找证据，再写代码。**



# 拒绝信息堆砌：按生命周期进行 Context 分层

📄 需求与观察层（临时 / Chat, Output）：  
当前任务、Diff、测试报告。阅后即焚或归档。

🔍 动作与证据层（中短期 / Tools, References）：  
可用工具、Schema、样例代码。

📄 规则与流程层（长期 / AGENTS.md, Skills）：  
项目规范、验收标准、高频任务 workflows。

⚠️ 底层安全边界（硬性约束 / Policy, opencode.json）：  
权限黑白名单、沙箱隔离配置。

不要把所有内容都塞进聊天框！

按需加载、分层管理。

全部堆砌在 Prompt 中会导致系统失忆、指令冲突和维护灾难。



# 把安全写进运行时：工具契约与红绿灯权限系统



## 工具契约 (Tool Contract)

要求：

- 输入 Schema 化
- 结构化且清晰的错误输出
- 设定的超时中断机制
- 动作可审计与回滚

限制动作不是限制智能。只在 Prompt 里写“不要读敏感文件”是软约束，真正的安全必须在运行时 (Runtime) 拦截。



# 成熟系统的底线：五层验证漏斗与 Git 主保险网



**Git 快照（主保险网）：git stash 或 git commit**  
**Agent 修改前的强制动作。成熟系统不是不犯错，而是错误发生后能够毫秒级回滚。**  
**不要把会话级的 Undo 当作唯一救命稻草。**



# 状态剥离：决不能让“对话框”承担所有正式状态

## Chat（控制面 / 临时）

用于协商、澄清、临时反馈。  
充满试探和错误，不应作为正式依据。

## Trace（过程记录 / 日志）

Agent 实际执行动作和 API 调用的  
审计流水。

## Workspace State（事实面 / 真实状态）

项目当前真实的文件、配置和代码结果  
(Artifacts)。

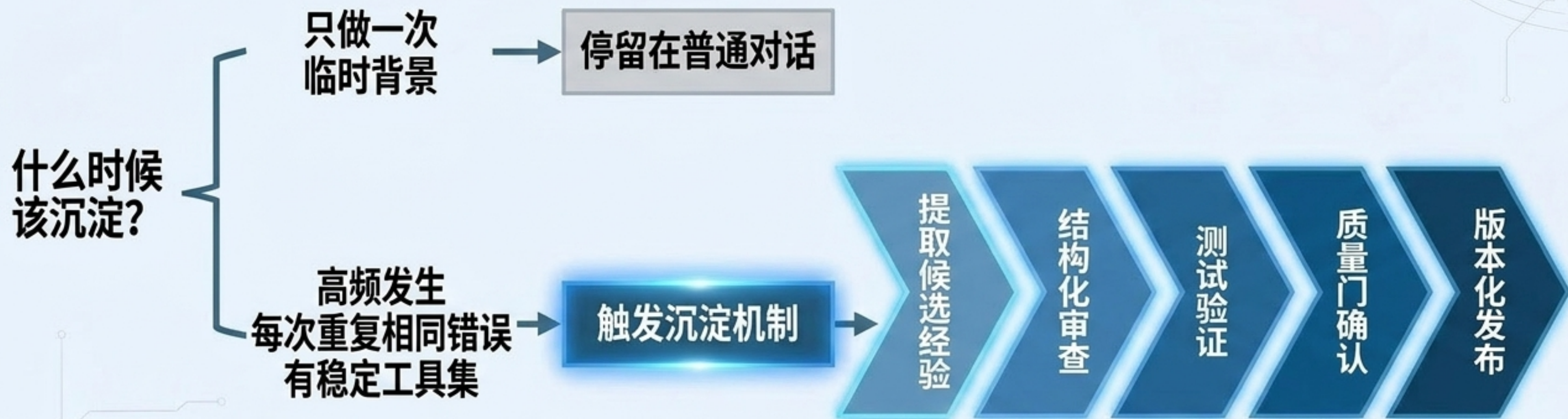
## Skill & Memory（长期经验 / 资产）

经过验证、可跨会话复用的标准操作流程。

**核心原则：聊天记录是一张草稿纸。  
让草稿纸归草稿纸，让工程资产归工程资产。**



# 经验的受控沉淀：从一次性对话演进为企业级 Skill



**警示：**“越用越好”不等于“自动把聊天记录写进记忆”。  
一次性偏好、未验证结论或恶意输入。能力沉淀需要类似软件发布的版本治理。



# 未来的应用架构：轻量级前端（Thin UI）协同后台 Agent Runtime

## Human UI（人机交互层）

- 输入意图（Chat/Form）
- 查看成果（Viewer）
- 比较 Diff 与纠错
- 审批接管

特征：界面变薄，剥离复杂的步骤操作。

## Agent Interface / Runtime（机器交互层）

- 加载 Workspace
- 发现能力与调用工具
- 管理权限沙箱
- 执行验证闭环

特征：深处后台，负责处理不确定性与执行闭环。



**洞察：**软件不会全变成一个聊天框。未来的形态是“表达意图的聊天 + 精确输入的表单 + 审批成果的控件”组成的混合工作系统。



# 核心精髓：人类定方向，AI 做执行，系统保稳定

- ✓ 先背景，后任务（不要让 AI 盲人摸象）
- ✓ 先 Plan，后 Build（先找证据再动刀）
- ✓ 先验证，后声明完成（拒绝幻觉性的“已修复”）
- ✓ 先 Checkpoint，后放手（Git 是你永远的底线）
- ✓ 先质量门，后自动成长（受控沉淀，拒绝污染）

终极思考：未来的核心竞争力不再是钻研 Prompt 奇技淫巧，  
而是为 AI 搭建一个目标明确、工具清晰、权限合理、且必定可验证的任务环境。